Department of Computer Science, University College London

COMP0227 Lab 03: Depth in Images and in SLAM

20th October, 2026

1 Scope and Purpose

In Lecture 02, we looked at depth as a per-pixel property. A pixel together with its depth gives a point in 3D. A single camera cannot measure that depth on its own: it needs a second view, and enough parallax between the two views. Instead, a network can be trained to predict the depth from one image, either up to an unknown scale or in metres. We also looked at the places in a monocular SLAM system where such a depth could be used. pySLAM includes several monocular depth networks. In this lab, we will measure how good they are, and then try the simplest of those places.

This lab has three parts:

1. Run the depth networks on *single* images and compare them. Which of them give believable metres, what do they get wrong, and how long do they take on your computer? At the end of this part you should be able to choose one network for the rest of the lab.
2. Run one network over a whole sequence whose camera also recorded a depth image, and score the prediction against that image, both as the network gives it and with its scale thrown away. Then read the predicted depth where SLAM needs it: at the feature points.
3. Use the predicted depth in SLAM in the simplest way, in place of the depth camera's. Compare monocular SLAM, SLAM with the camera's depth and SLAM with the predicted depth, with several runs of each.

This lab assumes that you have completed Labs 00, 01 and 02, and that pySLAM is installed and working. As before, run all the commands from the `pyslam` folder.

Predicting a whole sequence (Activity 4) takes from a few minutes on a GPU to most of an hour on a CPU with the large model. It does not depend on Part 1, so start it at the beginning of the lab, or use the small model.

2 Setting Up

Do this before the lab: the downloads are large.

First bring pySLAM up to the version for this lab, as the CHANGELOG describes. From the `COMP0227_26-27` **folder:**

```
git pull
git submodule update --init --depth 1 pyslam
cd pyslam
pixi run build
```

On native Windows the last command is `pixi run -e default-win build`**.** `build` **only does what has changed, and nothing you installed before is lost.**

The depth networks need the `depth` **level of pySLAM. This is a second pixi environment, as the** `semantics` **level was in Lab 02. The following command installs it, together with the code and the weights of the three networks:**

```
pixi run -e depth models-depth
```

The environment adds only about 60 MB, since it shares PyTorch with the default one, but the weights of the networks are about 7 GB. At the end, the command runs each network once and prints one line per network, which should say `OK`**. If it stops, run it again: whatever is already done is skipped.**

- **If you installed the** `semantics` **level in Lab 02, you already have the** `depth` **level, because each level includes the ones below it. Use** `-e semantics` **in place of** `-e depth` **in every command of this sheet, including the one above. pixi then installs no second environment.**
- **On Linux without an NVIDIA GPU, use** `-e depth-cpu` **(or** `-e semantics-cpu`**) instead, everywhere.**
- **If disk space is short,** `pixi clean cache` **deletes the copy of the downloaded packages which pixi keeps. The smallest Depth Anything V2 model (**`--size small` **below) is enough for everything in this lab.** `pixi run -e depth models-depth-small` **installs only that model, for indoor and for outdoor scenes (0.2 GB), instead of everything. Then add** `--size small` **to every Depth Anything V2 command of this sheet. Depth Pro and Depth Anything 3 can be left to other members of your group.**

Part 2 needs a sequence with a depth image for every frame. Download `rgbd_dataset_freiburg1_xyz` **(**`fr1_xyz` **below) from the TUM RGB-D benchmark web site (0.45 GB) and unpack it. You may already have it from Lab 01 (Activity 6). Its folder holds** `rgb/`**,** `depth/`**,** `rgb.txt`**,** `depth.txt` **and** `groundtruth.txt`**. Then, as in Lab 02 (Activity 7), write the file which pairs each colour image with its depth image:**

```
S=path/to/rgbd_dataset_freiburg1_xyz
pixi run python scripts/associate.py $S/rgb.txt $S/depth.txt \
    > $S/associations.txt
```

(`S` **is just a shorthand for the folder, and the commands below use it too.) The file lists 792 frames: of the 798 colour images, 6 have no depth image close enough in time.**

The depth images are 16-bit PNGs. Each pixel holds the depth multiplied by 5000: a value of 5000 is one metre, and a value of 0 means that the camera measured nothing there. The factor is `DepthMapFactor` **in** `settings/TUM1.yaml`**. As Lecture 02 showed, every dataset picks its own factor, and a wrong factor gives a map of the wrong size without any error message.**

The SLAM commands of Part 3 run in the default environment, as before.

On Native Windows

The `depth` level does not exist on native Windows. Use WSL2 for Parts 1 and 2, or let a member of your group predict the depth maps and copy them to you. Part 3 only reads the maps, so it runs at the default level, with `-e default-win` as in Lab 02. So does Activity 6 with `--load`. The PowerShell notes of Lab 02 also apply here. In particular, write an associations file with `| Out-File -Encoding ascii` instead of `>`.

3 The Methods

Table 1 lists the networks. The names are the ones pySLAM uses.

Name	Network	Gives	Uses the camera's focal length
`DEPTH_ANYTHING_V2`	**Depth Anything V2, metric version**	**depth in metres; one model for indoor scenes and one for outdoor scenes, each in three sizes**	**no**
`DEPTH_ANYTHING_V3`	**Depth Anything 3, metric model**	**depth in metres**	**yes: the one in the settings file, or an assumed one if there is none**
`DEPTH_PRO`	**Depth Pro**	**depth in metres**	**yes: the one in the settings file, or its own estimate if there is none**

Table 1: The depth networks of this lab. All three need the `depth` level. pySLAM has other depth methods, which need a stereo pair and are not used here.

As Tutorial 02 explained, the metric models of Depth Anything V2 were fine-tuned on synthetic images: the indoor model on rendered rooms, the outdoor model on rendered streets. pySLAM uses the indoor model for a TUM sequence, a photograph and the `desk` and `box` test images, and the outdoor model for the other test images and for a video; `--scene indoor` or `--scene outdoor` overrides it. A network trained to give metres gives them for the kind of scene, and the kind of camera, that it was trained on. Whether it does so on your images is one of the things you will measure. You can always throw the scale away and keep only the shape of the depth map.

The full list of methods, with what each one gives, is printed by:

```
pixi run -e depth depth-prediction --list
```

4 Part 1: Single Images

Activity 1: Predict the Depth of One Image

Run:

```
pixi run -e depth depth-prediction --method DEPTH_ANYTHING_V2 --test kitti
```

This predicts the depth of one frame of the KITTI video with the outdoor model of Depth Anything V2. It uses the large size unless you add `--size small` or `--size base`, which are faster and less accurate. On a CPU, add `--size small` throughout Part 1, and `--runs 1` to Depth Pro's command in Activity 2, since the timing repeats each prediction three times. The first run of a network is slow, because the model has to be loaded. Three windows open:

- `Image`**: the image.**
- `Depth`**: the predicted depth of every pixel, coloured from near (yellow) to far (dark blue), with a scale bar in metres. The title of the window gives the depth of the pixel under the mouse.**
- `3D`**: every pixel lifted into 3D with its predicted depth and the camera's calibration, as in Lecture 02. Drag with the mouse to turn the view, and scroll to zoom.**

Press `q` or Esc in the `Image` or `Depth` window, or close the `3D` window, to stop. The last line in the terminal is a summary:

```
Summary: DEPTH_ANYTHING_V2 on kitti (1226x370 pixels): metric, outdoor
model, large, on mps; 0.74 s per image (mean of 3 runs after the first,
which took 0.85 s; 5.3 s to load the model); depth 5.6 m to 79.5 m,
median 22.8 m
```

Your numbers, especially the times, will be different. `mps` is the GPU of a Mac; on other computers this reads `cuda` or `cpu`.

- **The road starts at the bottom of the image. Does its depth grow smoothly up the image? Where does the depth jump?**
- **Look at the edges of the cars and the poles in the `Depth` window. Are they sharp? On the pixels along an edge, is the depth that of the car, that of what is behind it, or something in between? Turn the view in `3D`: what do these pixels become?**
- **What depth does the network give the sky? Could any depth be right there? A depth network has to give every pixel a depth, just as a segmentation network had to give every pixel a class.**
- **In `3D`, do the fronts of the buildings come out flat and vertical? Does the road come out flat? Look along the road to its far end.**
- **The depth map holds the distance along the camera's axis (Z), not the distance along the ray through the pixel. Using the focal length and the principal point in `settings/KITTI04-12.yaml`, work out how much the two differ for a pixel at the left edge of the image, halfway up.**

Activity 2: Compare the Networks on the Same Image

Run the same image through the other two networks:

```
pixi run -e depth depth-prediction --method DEPTH_ANYTHING_V3 --test kitti
pixi run -e depth depth-prediction --method DEPTH_PRO --test kitti
```

Then fill in the table from the summary lines, reading the distance to the nearest car with the mouse in the `Depth` window. Depth Pro is by far the slowest: up to a few seconds per image on a GPU, and half a minute or more on a CPU. To save time, each member of your group can run one network and pool the numbers: it is the comparison between them that we are interested in.

	Depth: min, median, max (m)	Nearest car (m)	Time per image (s)	Load (s)
DEPTH_ANYTHING_V2				
DEPTH_ANYTHING_V3				
DEPTH_PRO				

- **All three networks were trained to give metres. Do they agree about the nearest car, and about the far end of the road? Which one would you believe, and what could you check it against?**
- **Look at the thin things: the poles, the tree trunks, the cyclist. Which network keeps them, and which one smears them into the background?**
- **Which network is the fastest, and which the slowest? The KITTI video has 10 frames per second. Which of the networks could keep up with it on your computer? Compare with the rest of your group: the summary line says which device each network ran on.**
- **Table 1 says which of the networks use the camera's focal length. Why would a network need the focal length to give metres? What would a focal length that is 10% too large do to Depth Pro's depth? What would it do to Depth Anything V2's?**

Activity 3: Indoors, Outdoors and the Wrong Model

Depth Anything V2 has two metric models, and `--scene` chooses between them (see The Methods). Run the desk image with each:

```
pixi run -e depth depth-prediction --method DEPTH_ANYTHING_V2 \
    --test desk --scene indoor
pixi run -e depth depth-prediction --method DEPTH_ANYTHING_V2 \
    --test desk --scene outdoor
```

- **Which model puts the desk at a believable distance? By roughly what factor is the other model wrong? Compare the medians in the two summary lines.**
- **Is the shape of the depth map wrong too (the order of near and far, the edges), or is only its scale wrong? Compare the two `Depth` windows side by side. Each one colours its own range of depths.**
- **Run `--test mars` with both models, and with Depth Pro. None of them was trained on Mars. What distances do they give, and how sure of itself does each depth map look? Like the segmentation networks of Lab 02, a depth network has no way of saying that it does not know.**

Then photograph something whose distance you can measure, such as a door, a desk or a wall (use a tape measure, or pace it out), and run the photograph with `--image my/photo.jpg`. A photograph has no settings file, so Depth Pro estimates the focal length itself and Depth Anything 3 assumes one. The summary line gives the focal length that each of them used. A photograph larger than 1600 pixels is first reduced, and the terminal says so. A focal length in pixels shrinks with the image, so compare focal lengths at the same width.

- **Which network gets closest to the distance you measured?**
- **If you made a settings file for your phone in Lab 01 (Activity 7), how close is Depth Pro's estimate to the focal length in that file? (With `--settings`, both networks use the file's focal length instead.)**

Table 2 lists the test images. Use `--test` to choose one, or `--image` with an image file of your own.

Name	The image
kitti	a frame of the KITTI 06 video (a street)
kitti_next	the frame half a second later
desk	a desk with objects on it (indoors)
churchill	a building
box	a box among other objects
mars	the surface of Mars

Table 2: The test images (the same as in Lab 02).

5 Part 2: A Sequence with a Depth Camera

Activity 4: Predict and Score a Whole Sequence

The same script predicts every frame of a sequence. When the sequence has depth images of its own, it also scores each prediction against them. Run it on `fr1_xyz`, and save the depth maps for Part 3:

```
pixi run -e depth depth-prediction --method DEPTH_ANYTHING_V2 \
    --tum $S --settings settings/TUM1.yaml --save dav2
```

On a CPU (if Activity 1's summary said `on cpu`), add `--size small`. The 792 frames take a few minutes on a GPU. On a CPU the large model takes most of an hour, and the small one about 5 minutes, with scores a little worse.

`--save dav2` writes two things inside the folder of the sequence, next to the camera's `depth/` folder:

- `depth_dav2/`: one predicted depth map per frame, in the camera's format, and the scores of every frame in `scores.csv` (Activity 5);
- `associations_dav2.txt`: the list of the frames, like `associations.txt`, except that it pairs each colour image with its *predicted* depth map.

To give the maps to another member of your group, copy these two items into their copy of the sequence.

If even the small model would take too long, `--step 5` predicts only every fifth frame. The scores are just as good, but SLAM in Part 3 would then see only every fifth frame, so get the maps from your group instead. `p` pauses and resumes; `q` or Esc stops.

The windows are those of Activity 1, now frame by frame, with one more: `Error`, which compares the prediction with the camera's depth once the prediction's scale has been removed (the shape-only score below). It is white where the two agree, red where the prediction is too far, blue where it is too near, and black where the camera has no depth; `m` switches to the depth as the network gives it. The title of the `Depth` window now also gives the camera's depth under the mouse. While the script runs, watch `Error`.

- Where is the prediction wrong: at the edges of the desk and of the monitor, on the far wall, on the floor, on the monitor's screen?
- Where are the black pixels, where the camera measured nothing? The camera projects a pattern of infrared light onto the scene: why would that fail there? Which depth would you believe at the edge of the desk, and which on a shiny or black surface?

When the script finishes, the summary line gives the mean time per frame and three scores, averaged over the frames. Here d is the predicted depth of a pixel and d^* the camera's, and only the pixels where the camera has a depth are counted:

- **AbsRel, the absolute relative error: the mean over the pixels of $|d - d^*|/d^*$.**
- **$\delta < 1.25$: the share of the pixels where $\max(d/d^*, d^*/d) < 1.25$, that is, where the prediction is within 25%.**
- **Median scale: the median over the frame of d^*/d. If the network gives metres, this is 1.**

Each of the first two is given twice: as the network gives it (*metric*), and after each predicted map has been multiplied by its own median scale (*shape only*). Note that only a scale is removed. The relative depths of Lecture 02, and the papers' scores (Tutorial 02), also remove a shift in inverse depth, which is more lenient, so their numbers cannot be compared with these.

```
Summary: DEPTH_ANYTHING_V2 on rgbd_dataset_freiburg1_xyz (792 frames,
640x480): indoor model, large, on mps; 0.24 s per frame; metric: AbsRel
0.26, delta<1.25 0.58, median scale 0.81; shape only: AbsRel 0.04,
delta<1.25 0.98; 792 depth maps saved to depth_dav2
```

Your numbers will be different.

- **Is the median scale 1? If it were 1.10, would the network's depth be too short or too long, and by how much at 2 m?**
- **How much better are the shape-only scores than the metric ones? What does the network get right: the shape of the scene, or its size?**
- **The camera's depth is the reference we score against, but it is not perfect either. Can you find a place where the network is right and the camera is wrong?**
- **Open one of the saved maps in `pixi run python`, with `cv2.imread(path, cv2.IMREAD_UNCHANGED)`. What value does a pixel hold where the network put the depth at 2 m? If `settings/TUM1.yaml` said `DepthMapFactor: 1000.0` instead of 5000, how large would SLAM think the room was in Part 3? Would anything warn you?**

Activity 5: Measure the Stability

Watching the `Error` window shows you where the prediction is wrong, but not how it changes from one frame to the next. To measure that, use the scores the script saved: `depth_dav2/scores.csv` has one line per predicted frame: the frame number, its colour image, the time, and the scores of Activity 4 (`absrel_metric`, `absrel_shape`, `delta_metric`, `delta_shape`, `median_scale`). Open it in a spreadsheet, or in Python:

```
import pandas as pd
import matplotlib.pyplot as plt
scores = pd.read_csv(
    "path/to/rgbd_dataset_freiburg1_xyz/depth_dav2/scores.csv")
scores.plot(x="frame", y=["absrel_metric", "absrel_shape"])
scores.plot(x="frame", y="median_scale")
plt.show()
```

(`pixi run python` opens a Python with pandas.)

- Plot the median scale against the frame number. If the network gave metres, the median scale would be 1 in every frame. Even if it is not 1, does it at least stay the same? How much does it change from one frame to the next, and over the whole sequence?
- Find the frames where `absrel_metric` is largest, and look at their images in `rgb/`. What is in them: an object close to the camera, the edge of the desk, a fast movement?
- Comparing the median scale of successive frames is a crude test of stability, because the camera moves and successive frames do not see exactly the same surfaces. What would a better test have to do?
- A SLAM system which took the network's metres in every frame would build its map from pieces of slightly different sizes. Of the places in SLAM where Lecture 02 suggested a learned depth could be used, which would suffer from this, and which would not?

If another member of your group predicts the sequence with another network (with another `--save` name; `--step 5` is enough here), compare the two `scores.csv` files. Is the more accurate network also the more stable one?

Activity 6: Depth at the Feature Points

SLAM does not use the depth of every pixel, only the depth at its keypoints. On this sequence these are 1000 ORB features in each frame, which sit on corners and edges, so it is the depth at those points which matters to SLAM.

Open the file `pyslam/depth_estimation/depth_sampling.py`. It contains the function

```
def depth_at_keypoints(depth, keypoints, method="nearest", patch=5):
```

Its docstring says what it is given and what it must return. In short, it gets a depth map (a float array of shape H $\times$ W, in metres, 0 where there is no depth), an N $\times$ 2 array of keypoint positions (u, v) in pixels, which need not be whole numbers, the name of a method, and a patch size. It must return an array of N depths, with 0 wherever no depth can be read. There are three methods:

- `nearest`: the depth of the pixel which the keypoint falls in;
- `bilinear`: the depth interpolated from the four pixels around the keypoint, using only those which have a depth;
- `median`: the median of the pixels with a depth in a `patch` $\times$ `patch` window centred on the keypoint.

`nearest` is written for you, as an example. It rounds each keypoint to the pixel it falls in, keeps it inside the image, and reads the depth there. Note that u is the column and v the row, so the depth at (u, v) is `depth[v, u]`. The other two methods return `None`.

Start with `nearest`. Run the script with `--keypoints`: it detects ORB features in each frame as SLAM does, calls the function on the predicted depth and on the camera's depth, and compares the two. `--load dav2` reads the maps you saved in Activity 4 instead of running the network again, so this takes seconds:

```
pixi run -e depth depth-prediction --tum $S --settings settings/TUM1.yaml \
    --load dav2 --keypoints nearest --step 5
```

The `Image` window now shows the keypoints, coloured by their error as in the `Error` window, and grey where the function gives no depth from the network or from the camera. The summary line adds the AbsRel at the keypoints, next to the AbsRel over the whole image:

```
Summary: depth_dav2 on rgbd_dataset_freiburg1_xyz (159 of 792 frames,
every 5, 640x480): saved maps of DEPTH_ANYTHING_V2 (metric, indoor model,
large); metric: AbsRel 0.25, delta<1.25 0.58, median scale 0.81; shape
only: AbsRel 0.04, delta<1.25 0.98; at the keypoints (nearest): AbsRel
0.271 metric, 0.055 shape only (862 of 1000 keypoints per frame with
both depths)
```

- **Is the error at the keypoints larger or smaller than over the whole image? ORB keypoints sit on corners and edges. Is that where a depth network is at its best, or at its worst?**
- **Which keypoints have the largest errors? Pause and look at them. What is behind them?**

Now write `bilinear` and `median`. You need about fifteen lines. A loop over the keypoints is fine here, since there are only 1000 in a frame. Write `median` first: it is the simpler of the two. Useful functions: `np.floor` and the four weights for `bilinear`; `np.median` over a slice of the array for `median`, keeping only the values above zero; and `np.clip` to stay inside the image at its borders, as `nearest` does. Then run the script again with `--keypoints bilinear` and with `--keypoints median` (try `--patch 3` and `--patch 9` too). If a method still returns `None`, a warning says so and the keypoints are skipped.

- **Which method is best? What does each one do on an edge, where half of the pixels belong to the object and half to what is behind it? And at a keypoint where the camera's depth has a hole?**
- **If you had to give SLAM one depth per keypoint, which of the three methods would you use, and why?**

6 Part 3: Predicted Depth in SLAM

Lecture 02 described several places where a learned depth could enter monocular SLAM. The simplest one needs no change to pySLAM at all: give it the predicted depth maps as if a depth camera had measured them, and run it as RGB-D SLAM. This is called *pseudo RGB-D*. SLAM then treats the predicted depth as if it had been measured, so whatever the network gets wrong goes straight into the map.

Activity 7: SLAM with the Predicted Depth

As Lab 01 explained, `--tum` reads the list of the frames from `associations.txt` if there is one, and then runs RGB-D SLAM with the depth images it names. `--associations` names another list. So the predicted depth needs nothing but the list which Activity 4 wrote:

```
pixi run slam --tum $S --settings settings/TUM1.yaml \
    --associations associations_dav2.txt
```

Watch it run, as in Lab 00. Then run the same sequence with the camera's depth, and as monocular SLAM:

```
pixi run slam --tum $S --settings settings/TUM1.yaml
pixi run slam --tum $S --settings settings/TUM1.yaml --sensor mono
```

Each run takes a minute or two (about 45 seconds on a fast computer). pySLAM has a known fault which now and then freezes a run: if the frames stop advancing, press Ctrl-C (on native Windows, close the terminal window) and run the command again.

- `Map Viewer`**: with a depth image, the map is there in the first frame. Monocular SLAM has to wait until the camera has moved: the terminal line** `map: initialized with kfs 0, ...` **says in which frame it built its first map. On this sequence that is quick. Why does it have to wait at all, and what would make it wait longer? (Activity 9 has such a sequence.)**
- **Compare the pseudo RGB-D map with the RGB-D map. Look at the edges of the desk and of the monitor: are there points floating in front of them, or behind them? Where do those points come from? (Activity 6 may tell you.)**
- **The timing window: does reading a depth image change the time per frame? Where did the network's time go?**
- **At the end of each run, the trajectory error is printed. For the monocular run, pySLAM fits the estimated trajectory to the ground truth with a rotation, a translation and a scale. For the two RGB-D runs, it fits a rotation and a translation only (Lecture 02). Why does it do this? Which of the three numbers can you compare with which?**

Activity 8: Does It Help?

As you saw in Lab 01, two runs with the same settings give different errors, so one run is not a fair comparison. Run the three configurations three times each, without windows:

```
pixi run slam-evaluation --tum $S --settings settings/TUM1.yaml \
    --predicted-depth associations_dav2.txt --runs 3
```

This is nine runs: 5 to 10 minutes on a fast computer, and longer on a laptop. While it runs, go back to the questions of Activity 6, or start the download for Activity 9. If a run freezes, the evaluation stops it and runs it again by itself. The report (`report.html` **in the folder** `results/eval_<date>`**) has three columns,** `mono`**,** `rgbd` **and** `pseudo-rgbd`**, with the tables of Lab 01:** `rmse`**,** `max`**,** `percent_tracked` **and** `percent_lost`**.**

The `rmse` **of each column is computed as in Activity 7: with a scale for** `mono`**, and without one for the other two columns. To compare the three on the same footing, compute both errors for every run:**

```
pixi run python scripts/trajectory_errors.py results/eval_<date>
```

The script re-fits every run's trajectory to the ground truth twice, without and with a scale, and prints both errors and the scale it fitted. It also works on the `results/metrics_<date>` **folder of a single run of Activity 7. For the first monocular run, it printed:**

```
mono/rgbd_dataset_freiburg1_xyz/iteration_0: 0.051 m without scale;
0.011 m with scale (fitted scale 1.36)
```

The error with scale is the one in the report's `mono` **column. The error without scale is the one to compare with the other two columns.**

- **Read** `percent_tracked` **before** `rmse`**. Did any configuration lose track? A run which lost track can have a small error over the frames which it kept.**

- **The three runs of a configuration are the same command on the same frames. Did they give the same error, and the same trajectory? If not, where does the difference come from, and how many runs would you need before you believed a difference between two configurations?**
- **With both errors for every run: is the pseudo RGB-D configuration closer to the monocular one, or to the RGB-D one? Is the difference larger than the spread between runs of the same configuration?**
- **What scale was fitted to the pseudo RGB-D runs? If the map were in metres, it would be 1. Where did the map's scale come from? Compare it with the median scale of Activity 4: should the two agree?**
- **What scale was fitted to the monocular runs, and what decided it?**
- **From Activity 5 you know how much the network's scale changed along the sequence. Would you expect to see that in the trajectory, and where?**
- **On this sequence, did the network's depth help monocular SLAM, or not? What would you have to measure to say when it helps and when it does not?**

7 Extra Activities

These are optional.

Activity 9: A Harder Sequence

`rgbd_dataset_freiburg1_desk` **is the same camera in the same office as** `fr1_xyz`**, moved quickly over a cluttered desk. It is a harder case for monocular SLAM, which loses track when the camera turns quickly and there is little parallax. Download it from the TUM RGB-D web site (0.34 GB), point** `S` **at its folder, write its** `associations.txt` **as in Setting Up (it lists 573 frames), predict it with** `--save dav2` **as in Activity 4, and run the evaluation of Activity 8 on it.**

- **Read** `percent_tracked` **first. Did the monocular runs lose track? Did the predicted depth keep it?**
- **Are the scores of the prediction worse than on** `fr1_xyz`**? Is the difference in the shape or in the scale? What in this sequence would make a depth network's job harder: the speed, the clutter, or the objects close to the camera?**
- **This is the kind of sequence on which a learned depth should help most. Does the error table agree?**

Activity 10: Your Own Video

If you recorded a video of your own in Lab 01 (Activity 7), predict its depth with the settings file you made for it:

```
pixi run -e depth depth-prediction --method DEPTH_ANYTHING_V2 \
    --video my/video.mp4 --settings settings/MY_CAMERA.yaml --step 10
```

There is no depth to score against, so the summary line gives only the time per frame and the median predicted depth. A video gets the outdoor model of Depth Anything V2: if you filmed indoors, add `--scene indoor`**.**

- **Find a frame in which you know a distance, for example to a wall or a door. Pause, and read the depth under the mouse in the `Depth` window. How far off is it? Run it again with the other `--scene`: how much does the distance change?**
- **If you walk towards a wall in your video, does the depth of the wall shrink at the speed at which you walked?**

Running SLAM on your video with the predicted depth needs the video in a form pySLAM can run.

Activity 11: Depth and the Mask Together

Lab 02 gave the feature detector a mask made from the label maps, and the mask and the predicted depth can be used together. Segment `fr1_xyz` with DeepLabV3 as in Lab 02 (Activity 7), and run pseudo RGB-D with the monitor masked:

```
pixi run semantic-segmentation --method DEEPLABV3 --tum $S \
    --save labels/fr1_xyz
pixi run slam --tum $S --settings settings/TUM1.yaml \
    --associations associations_dav2.txt \
    --semantic-mask labels/fr1_xyz --mask-classes "tv monitor"
```

The mask comes from your function `mask_from_labels()` of Lab 02 (Part 3), in the same `pyslam` folder. If it still returns `None`, the run says that no frame was masked, and the results are those of Activity 7. The segmentation takes about a minute on a fast computer; on a CPU add `--step 5`, as in Lab 02. DeepLabV3 labels about 12% of the pixels as `tv monitor`.

- **Activity 6 showed which keypoints have the worst predicted depth. Are many of them on the monitor? Add the same two options, `--semantic-mask labels/fr1_xyz --mask-classes "tv monitor"`, to the evaluation of Activity 8. It then runs each of its three configurations a second time with the mask (`mono+mask`, `rgbd+mask`, `pseudo-rgbd+mask`): 18 runs, about 15 minutes on a fast computer. Does masking change the error? Is the change larger than the spread between runs?**
- **A label map says which pixels belong to a monitor, but not to which monitor (Lecture 02, second hour). Does that matter for this mask? When would it matter?**